



Instituto Tecnológico de Puerto Rico

Recinto de San Juan

LABORATORIO 6 - SEMANA 7

Estructuras y uniones | Del programa en consola a una interfaz visual

Curso	TESI 2105 - Programación Orientada a Objetos
Semana	7
Actividad	Laboratorio 6
Tema	Estructuras y uniones
Valor	50 puntos
Evidencia	Programa de consola + interfaz PyUIBuilder/Tkinter + explicación de conceptos
Propósito: aplicar declaración/agrupación de datos, acceso a miembros y el concepto de unión en un ejemplo sencillo; después visualizar la misma lógica mediante una GUI básica sin adelantar clases y objetos.	
Nombre del estudiante	Tairon A. Jaime Maritnez
Repositorio / carpeta	_____
Fecha	_____
Sección	_____

1. PARTE A - PROGRAMA INICIAL EN PYTHON

Crea un archivo llamado semana7_estructuras_union.es.py. Escribe y ejecuta el siguiente programa. No pases a PyUIBuilder hasta que el programa funcione en la terminal.

```
equipo = {
    "nombre": input("Nombre del equipo: "),
    "ram": int(input("RAM en GB: ")),
    "estado": input("Estado: "),
}

dato alternativo = {
    "tipo": input("Tipo de identificador (serie/activo): "),
    "valor": input("Valor del identificador: "),
}

print("\nREGISTRO DEL EQUIPO")
print("Nombre:", equipo["nombre"])
print("RAM:", equipo["ram"], "GB")
print("Estado:", equipo["estado"])
print("Identificador:", dato alternativo["tipo"], "=", dato alternativo["valor"])
```

Prueba requerida: ejecuta el programa con datos distintos al ejemplo del profesor. Conserva una captura de la ejecución.

2. PARTE B - IDENTIFICACIÓN DE LOS CONCEPTOS

Pregunta	Respuesta del estudiante
¿Qué entidad representa la estructura equipo?	Representa un objeto o un equipo físico.
¿Cuáles son sus tres miembros/campos?	nombre, ram y estado.
Escribe una expresión que acceda al campo ram.	equipo["ram"]
¿Qué representa dato alternativo?	Representa como un dato opcional.
¿Qué regla del concepto de unión intenta modelar tipo + valor?	intenta modelar la idea de que solo uno de varios tipos de datos puede estar presente o ser relevante en un momento dado, aunque en python se esté simulando.

Explica por qué esta representación no es una union nativa de memoria de C/C++.

Porque C++ comparte la misma ubicación de memoria mientras que en python, es como un diccionario que simplemente agrupa pares clave-valor en una memoria dinámica sin tener que compartir espacio de almacenamiento entre los campos.

3. PARTE C - PREPARACIÓN DE PYUIBUILDER

No necesitas instalar PyUIBuilder para esta actividad; utiliza la versión web indicada por el profesor. Trabaja en Chrome o Edge y mantén tkinter como framework.

1. Abre PyUIBuilder y verifica que el framework seleccionado sea tkinter.
2. En la ventana principal coloca un Label y un Entry para cada dato: Nombre, RAM, Estado, Tipo de identificador y Valor del identificador.
3. Añade un Button con texto Registrar.
4. Añade un Label con texto Resultado.
5. Acomoda la interfaz de forma clara. La apariencia no se evalúa por diseño avanzado; solo debe ser legible y funcional.
6. Antes de exportar, verifica que todos los controles estén colocados.
7. Usa Código de exportación para descargar el archivo .py. No uses la opción Exportar como clases.

Importante: completa el diseño antes de exportar. Después de modificar el archivo en VS Code, evita volver a exportar porque PyUIBuilder puede sobrescribir tus cambios manuales.

4. PARTE D - VERIFICACIÓN DEL CÓDIGO EXPORTADO

8. Abre el archivo .py exportado en Visual Studio Code.
9. Ejecuta el archivo sin modificarlo y comprueba que la ventana abre.
10. Localiza las líneas de los cinco Entry, el Button y el Label de Resultado.
11. Anota los nombres reales que PyUIBuilder asignó a esos widgets.

Widget	Nombre generado en tu código
Entry - Nombre	entry
Entry - RAM	entry1
Entry - Estado	entry2
Entry - Tipo	entry3
Entry - Valor	entry4
Button - Registrar	registrar
Label - Resultado	resultado

5. PARTE E - INTEGRACIÓN DE LA LÓGICA

Después del último widget creado y antes de `main.mainloop()`, añade una función siguiendo este modelo. Sustituye `entry`, `entry1`, etc. y `label_resultado` por los nombres reales de tu archivo.

```
def registrar_equipo():
    nombre = entry.get()
    ram = entry1.get()
    estado = entry2.get()
    tipo = entry3.get()
    valor = entry4.get()

    equipo = {
        "nombre": nombre,
        "ram": ram,
        "estado": estado
    }
    dato_alternativo = {"tipo": tipo, "valor": valor}

    texto = (
        f"{equipo['nombre']} | {equipo['ram']} GB | {equipo['estado']} | "
        f"{dato_alternativo['tipo']}: {dato_alternativo['valor']}"
    )
    label_resultado.config(text=texto)

button.config(command=registrar_equipo)
```

Si el texto de resultado se corta, localiza el Label de resultado y realiza el mismo ajuste probado en clase:

```
label_resultado.configure(anchor="w")
label_resultado.place(x=..., y=..., width=300, height=40)
```

Conserva las coordenadas x e y que generó tu diseño; modifica principalmente anchor y width.

6. PARTE F - PRUEBAS REQUERIDAS

Prueba	Qué debes comprobar	Resultado
1	La ventana abre sin errores.	Se observó que la ventana abrió sin errores alguno.
2	Escribes todos los datos y el botón responde.	Se puede visualizar que el botón si responde.
3	El resultado muestra nombre, RAM y estado.	Se puede visualizar el nombre, ram y estado.
4	El resultado muestra tipo y valor del identificador.	Si muestra el tipo y el valor identificado.
5	El texto no se corta en el Label de resultado.	El texto se cortaba antes, después de configurarlo según la instrucción se pudo observar el resultado completo.

7. EVIDENCIAS QUE DEBES SOMETER

- Archivo semana7_estructuras_uniones.py funcionando en consola.
- Archivo .py final de la interfaz exportada y modificada.
- Captura de la ejecución en consola.
- Captura de la interfaz mostrando un registro completo.

Main Window

File Edit New Item

Nombre: PC-1

RAM: 32

Estado: disponible

Tipo: activo

Valor: 300

Registrar

PC-1 | 32 GB | disponible | activo: 300

-
- Respuestas de la Parte B y tabla de nombres generados de la Parte D.

- Enlace al repositorio Git/GitHub o ubicación indicada por el profesor, si corresponde al flujo del curso.

8. PREGUNTAS DE CIERRE

12. ¿Qué ventaja tuvo agrupar los datos del equipo en una sola estructura?
Permite organizar mejor la información y manejar todo el equipo como un solo conjunto.
13. ¿Qué diferencia observas entre acceder a un miembro de la estructura y escribir directamente variables independientes?
Con la estructura se accede a cada dato de forma ordenada, por ejemplo equipo["ram"]. Con variables independientes, los datos quedan separados y el código se vuelve menos claro.
14. ¿Qué representa la unión en este laboratorio y cuál es la limitación del modelo utilizado en Python?
Representa un dato alternativo de identificación. La limitación es que en Python solo se simula la idea de unión con un diccionario; no funciona como una unión real de memoria en C/C++.
15. ¿Qué cambió al pasar de la terminal a la interfaz? ¿Cambió la lógica o cambió principalmente la forma de entrada/salida?
Cambió principalmente la forma de entrada y salida. La lógica del programa es la misma.

9. LÍMITES Y POLÍTICA DE ENTREGA

No se requiere utilizar clases, objetos, constructores, herencia, bases de datos ni múltiples ventanas. Esos temas no forman parte del alcance oficial de la Semana 7.

Toda entrega posterior a la fecha límite tendrá una penalización inicial de 10 puntos. Por cada día adicional de atraso después del primer día fuera de término, se descontarán 5 puntos adicionales.